

## ✖ Banking Sector SVM Example

### Objective

This notebook shows a **basic Support Vector Machine (SVM / SVC)** classification example with visualization.

### Scenario

A telecom/banking company wants to classify customers as Basic or Premium based on data usage and monthly bill.

### Target Column

Customer\_Type

### Class Meaning

0 = Basic Customer, 1 = Premium Customer

## ✖ Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

## ✖ Step 2: Create Simple Dataset

```
df = pd.DataFrame({
    "Data_Usage_GB": [2, 5, 8, 10, 12, 15, 18, 22, 25, 28, 30, 35],
    "Monthly_Bill": [1000, 1200, 1500, 1800, 2500, 2700, 3200, 4000, 4500, 4800, 5000, 5500],
    "Customer_Type": [0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1]
})

df
```

|    | Data_Usage_GB | Monthly_Bill | Customer_Type |
|----|---------------|--------------|---------------|
| 0  | 2             | 1000         | 0             |
| 1  | 5             | 1200         | 0             |
| 2  | 8             | 1500         | 0             |
| 3  | 10            | 1800         | 0             |
| 4  | 12            | 2500         | 1             |
| 5  | 15            | 2700         | 1             |
| 6  | 18            | 3200         | 1             |
| 7  | 22            | 4000         | 1             |
| 8  | 25            | 4500         | 1             |
| 9  | 28            | 4800         | 1             |
| 10 | 30            | 5000         | 1             |
| 11 | 35            | 5500         | 1             |

## ✖ Step 3: Split Features and Target

**X** contains input features.

**y** contains the output/class column.

```
X = df[['Data_Usage_GB', 'Monthly_Bill']]
y = df['Customer_Type']

print("Features:")
X.head()
```

```
print("\nTarget:")
y.head()
```

Features:

Target:

|   | Customer_Type |
|---|---------------|
| 0 | 0             |
| 1 | 0             |
| 2 | 0             |
| 3 | 0             |
| 4 | 1             |

dtype: int64

## Step 4: Train-Test Split

Training data is used to train the model.

Testing data is used to check model performance.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

```
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 9
Testing samples: 3
```

## Step 5: Train SVM Model

**SVC** means **Support Vector Classifier**.

We use `kernel='linear'` because it is simple and easy to visualize.

```
model = SVC(kernel='linear')
model
```

▼ SVC ⓘ ?  
SVC(kernel='linear')

```
model = SVC(kernel='linear')
model.fit(X_train, y_train)
```

```
print("SVM model trained successfully.")
```

```
SVM model trained successfully.
```

```
model.fit(X_train, y_train)
```

▼ SVC ⓘ ?  
SVC(kernel='linear')

## Step 6: Make Predictions

```
y_pred = model.predict(X_test)
```

```
result = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})
```

```
result
```

|   | Actual | Predicted |
|---|--------|-----------|
| 0 | 1      | 1         |
| 1 | 1      | 1         |
| 2 | 0      | 0         |

## Step 7: Evaluate Model

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Accuracy: 1.0

Confusion Matrix:

```
[[1 0]
 [0 2]]
```

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 1       |
| 1            | 1.00      | 1.00   | 1.00     | 2       |
| accuracy     |           |        | 1.00     | 3       |
| macro avg    | 1.00      | 1.00   | 1.00     | 3       |
| weighted avg | 1.00      | 1.00   | 1.00     | 3       |

## Step 8: SVM Visualization

Because we selected only **two features**, we can draw the SVM decision boundary.

```
X_values = X.values
y_values = y.values

visual_model = SVC(kernel='linear')
visual_model.fit(X_values, y_values)

x_min, x_max = X_values[:, 0].min() - 1, X_values[:, 0].max() + 1
y_min, y_max = X_values[:, 1].min() - 100, X_values[:, 1].max() + 100

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 200),
    np.linspace(y_min, y_max, 200)
)

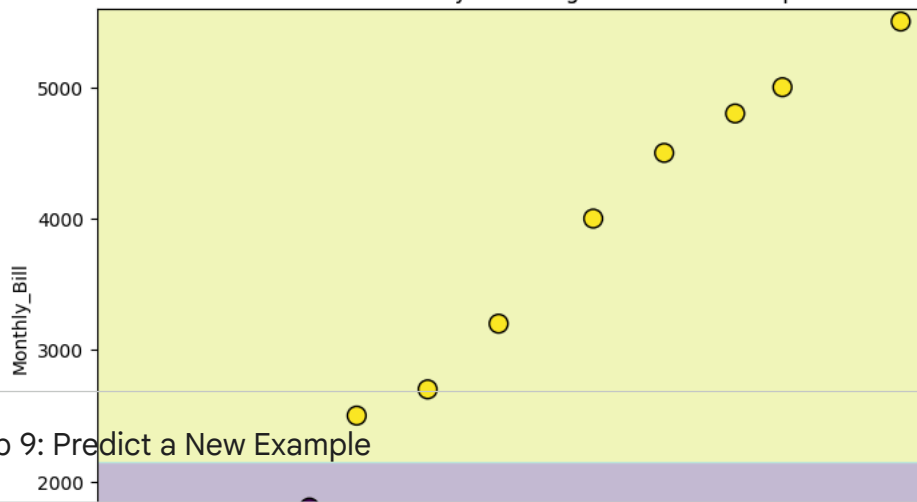
Z = visual_model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3)

plt.scatter(
    X_values[:, 0],
    X_values[:, 1],
    c=y_values,
    edgecolors='k',
    s=100
)

plt.xlabel("Data_Usage_GB")
plt.ylabel("Monthly_Bill")
plt.title("SVM Decision Boundary - Banking Sector SVM Example")
plt.show()
```

SVM Decision Boundary - Banking Sector SVM Example



## Step 9: Predict a New Example

```
new_customer = [[18, 3200]]

prediction = model.predict(new_customer)

print("Prediction:", prediction[0])

if prediction[0] == 1:
    print("Customer is likely Premium.")
else:
    print("Customer is likely Basic.")
```

```
Prediction: 1
Customer is likely Premium.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but SVC was fit with feature names
  warnings.warn(
```

## Final Note

This is a **basic educational example**.

For real-world projects, use:

- larger datasets
- feature scaling
- GridSearchCV
- cross-validation
- more evaluation metrics